



Universidad de
SanAndrés

I404 - Aprendizaje reforzado

Proyecto Final Aprendizaje Reforzado

Basso, Ignacio Carlos
Ostrovsky, Eliana

30 de noviembre de 2025

Ingeniería en Inteligencia Artificial

ÍNDICE

I.	Objetivos	1
II.	Introducción	1
II-A.	Marco Teórico y Dinámica de Proximal Policy Optimization (PPO)	1
II-B.	Entorno de Simulación: Unity y ML-Agents	2
II-C.	Definición del Proceso de Decisión de Markov (MDP)	2
III.	Desarrollo	2
III-A.	Configuración del Entorno y Desafíos de Legacy	2
III-B.	Implementación y Optimización del Algoritmo PPO-Clip	3
III-C.	Validación en Entornos Estándar (Gymnasium)	3
III-D.	El "Bridge"TCP: Implementación y Configuración de Componentes	4
III-E.	Dead End: La Ilusión del Enjambre y el Ajuste de Recompensas	4
III-F.	Migración Estratégica: Abandono del Bridge TCP e Integración con ML-Agents	4
III-G.	Revisión y Rediseño del Sistema de Recompensas	5
III-H.	Validación Preliminar en Entornos Sintéticos	6
III-I.	Validación en Entornos de Control Clásicos y Continuos	7
III-J.	Benchmarking: Comparativa con Stable Baselines 3	8
III-K.	Análisis de la Curva de Entrenamiento en Unity	9
IV.	Conclusión y Trabajo Futuro	10
Apéndice		11
A.	Circuito	11

I. OBJETIVOS

El objetivo general del presente trabajo es la implementación, entrenamiento y evaluación de un agente de Aprendizaje por Refuerzo Profundo (Deep Reinforcement Learning) capaz de controlar un vehículo autónomo en un entorno de simulación basado en Unity.

El proyecto toma como punto de partida el entorno de simulación desarrollado originalmente por Samuel Arzt para la demostración de Algoritmos Genéticos y Redes Neuronales Evolutivas (EANNs), disponible en https://github.com/ArztSamuel/Applying_EANNs. La meta principal consiste en sustituir la lógica de control evolutiva original por un enfoque de Aprendizaje por Refuerzo, específicamente utilizando el algoritmo *Proximal Policy Optimization* con *Clipping* (PPO-Clip). Se busca demostrar la viabilidad y eficiencia de PPO para resolver problemas de control continuo en entornos de conducción con físicas simuladas.

Para alcanzar este propósito general, se han definido los siguientes objetivos particulares:

- **Interconexión de Entornos (Bridge):** Desarrollar una interfaz de comunicación robusta mediante protocolo TCP/IP que vincule el motor de física de Unity (C#) con un entorno de entrenamiento externo en Python. Esto implica la adaptación del *loop* de simulación para permitir el envío de observaciones (lecturas de sensores raycast y velocidad) y la recepción de acciones continuas (dirección y aceleración) en tiempo real.
- **Navegación y Evasión de Obstáculos:** Entrenar al agente para que interprete correctamente las señales de los 5 sensores de distancia frontales, permitiéndole mantenerse dentro de los límites de la pista y evitar colisiones contra las paredes de los carriles, maximizando así la duración de los episodios.
- **Optimización de Velocidad y Tiempo:** Lograr que el agente aprenda a administrar la aceleración para recorrer el circuito en el menor tiempo posible, manteniendo el control del vehículo a altas velocidades sin desestabilizarse.
- **Optimización de Trayectoria (Racing Line):** Como objetivo avanzado, se busca que la política aprendida no solo evite choques, sino que descubra la trayectoria óptima de carrera. Esto implica que el agente aprenda a tomar las curvas cerradas (“cortar camino”) para minimizar la distancia total recorrida, un comportamiento emergente que denota una comprensión profunda de la dinámica del entorno y la función de recompensa.

II. INTRODUCCIÓN

El aprendizaje por refuerzo (RL, por sus siglas en inglés) ha emergido como una de las herramientas más potentes para resolver problemas de control continuo y toma de decisiones secuenciales. A diferencia del aprendizaje supervisado, donde se dispone de un conjunto de datos etiquetado, en RL un agente debe aprender interactuando con un entorno dinámico, buscando maximizar una señal de recompensa acumulada a lo largo del tiempo.

El presente trabajo se centra en la aplicación de técnicas de RL Profundo (Deep RL) para la conducción autónoma. El objetivo central es dotar a un vehículo simulado de la capacidad de navegar un circuito complejo, optimizando no solo la evasión de obstáculos, sino también la velocidad y la trayectoria de carrera (*racing line*). Para ello, se sustituye el enfoque clásico de algoritmos genéticos y neuroevolución, presente en el proyecto base de Samuel Arzt, por un agente entrenado mediante el algoritmo *Proximal Policy Optimization* (PPO).

A continuación, se detallan los fundamentos teóricos del algoritmo seleccionado y las características del entorno de simulación.

II-A. Marco Teórico y Dinámica de Proximal Policy Optimization (PPO)

Los métodos de *Policy Gradient* buscan optimizar directamente una política parametrizada $\pi_\theta(a|s)$ mediante ascenso de gradiente sobre una función objetivo $J(\theta)$. Algoritmos como REINFORCE utilizan la función de ventaja para actualizar los parámetros de la política incrementando la probabilidad de acciones que obtuvieron alta recompensa. Sin embargo, estos métodos suelen presentar alta varianza y sensibilidad a la tasa de aprendizaje, lo que puede llevar a inestabilidad y actualizaciones destructivas de la política.

Para abordar estas limitaciones, *Trust Region Policy Optimization* (TRPO) introduce restricciones basadas en la divergencia KL entre políticas para asegurar que las actualizaciones sean seguras. Si bien TRPO ofrece estabilidad teórica fuerte, su implementación es costosa debido a la necesidad de cálculos de segundo orden.

En este contexto, se selecciona **Proximal Policy Optimization con Clipping (PPO-Clip)**. PPO conserva la idea de restringir el cambio entre políticas, pero reemplaza el cálculo de segundo orden por una función objetivo más simple que aplica un mecanismo de recorte (*clipping*). Esto permite estabilizar el entrenamiento sin sacrificar eficiencia computacional ni simplicidad de implementación.

La función objetivo de PPO-Clip se define como:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}(s_t, a_t), \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}(s_t, a_t) \right) \right], \quad (1)$$

donde $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ es el cociente de probabilidades, $\hat{A}(s_t, a_t)$ es la ventaja estimada y ϵ es el umbral de recorte (típicamente 0.2). El término de recorte penaliza explícitamente actualizaciones excesivamente grandes en la política, asegurando estabilidad durante el entrenamiento.

Dinámica del Ratio de Probabilidades y Estabilidad: El ratio $r_t(\theta)$ mide la magnitud del cambio entre la política nueva y la política anterior. PPO evita que este valor crezca demasiado mediante:

- **Clipping:** restringe el ratio al intervalo $[1 - \epsilon, 1 + \epsilon]$, anulando gradientes que conducirían a pasos destructivos.
- **Optimización por épocas con mini-batches:** PPO reutiliza los mismos rollouts para múltiples actualizaciones, maximizando la eficiencia de muestras sin perder estabilidad gracias al clipping.

El esquema general de actualización es:

Listing 1: Ciclo de actualización PPO (representativo)

```
for epoch in range(K_epochs):
    for batch in minibatches:
        ratio = new_policy / old_policy
        surr1 = ratio * advantage
        surr2 = torch.clamp(ratio, 1-eps, 1+eps) * advantage
        loss = -torch.min(surr1, surr2)
```

Este mecanismo resulta especialmente valioso en entornos continuos y sensibles, como la conducción autónoma. Al limitar explícitamente la magnitud de las actualizaciones, PPO previene oscilaciones bruscas que podrían llevar a comportamientos erráticos, permitiendo que el agente aprenda gradualmente estrategias efectivas incluso en zonas con alta complejidad dinámica, como curvas cerradas donde las políticas sin restricciones tienden a divergir.

II-B. Entorno de Simulación: Unity y ML-Agents

El ambiente de aprendizaje se ha construido sobre el motor gráfico **Unity**, aprovechando su motor de física para simular de manera realista la fricción, colisiones y dinámica vehicular. Se tomó como base el entorno de demostración de *EANNs* de Samuel Arzt.

A diferencia de la implementación original, donde la lógica de control residía internamente en scripts de C# operando bajo lógica evolutiva, en este proyecto se ha desacoplado el “cerebro” del agente. Se desarrolló una interfaz de comunicación TCP/IP personalizada (un *Bridge*) que permite conectar la simulación en Unity con un entorno de entrenamiento externo en Python (utilizando librerías estándar de Deep Learning como gymnasium, socket y json). Esto permite controlar el ciclo de física paso a paso, enviando observaciones y recibiendo acciones del agente PPO en tiempo real.

II-C. Definición del Proceso de Decisión de Markov (MDP)

El problema de control se modela formalmente como un MDP definido por la tupla (S, A, R, γ) :

- **Espacio de Estados (S):** El agente percibe el entorno a través de un vector de características compuesto por las lecturas de 5 sensores de distancia tipo *raycast* (lidar simulado) que barren el frente del vehículo, más la velocidad escalar actual del mismo.
- **Espacio de Acciones (A):** El agente opera en un espacio continuo de dos dimensiones: [dirección, aceleración], donde cada valor se encuentra normalizado en el rango $[-1, 1]$.
- **Función de Recompensa (R):** Se ha diseñado una función de recompensa densa que incentiva el progreso a lo largo de la pista (basado en *checkpoints*), penaliza las colisiones contra las paredes y premia el mantenimiento de altas velocidades, fomentando así el comportamiento de carrera deseado. La misma otorga -10 cuando el vehículo choca, -5 por no moverse y $+1$ por completar el *track*.

III. DESARROLLO

El desarrollo del proyecto se llevó a cabo en etapas iterativas, enfrentando desafíos tanto en la ingeniería de software (integración Unity–Python) como en la implementación algorítmica del agente de Aprendizaje por Refuerzo. A lo largo del proceso también surgieron dificultades adicionales relacionadas con la compatibilidad entre sistemas operativos, particularmente entre macOS y Linux, lo cual afectó la ejecución de dependencias, librerías y entornos de desarrollo. A continuación, se detalla la cronología de los hechos, los obstáculos superados y las decisiones técnicas tomadas.

III-A. Configuración del Entorno y Desafíos de Legacy

La etapa inicial consistió en la puesta en marcha del entorno de simulación. El proyecto base seleccionado, desarrollado originalmente por Samuel Arzt, data de hace aproximadamente 8 años. Esto presentó una barrera de entrada significativa, ya que gran parte del código C# utilizaba funciones obsoletas (*deprecated*) y no era compatible con las versiones actuales del motor Unity.

Fue necesario realizar una refactorización extensa de los scripts del simulador y dedicar tiempo considerable a la curva de aprendizaje de la interfaz de Unity, la cual carecía de documentación para este proyecto específico. Esta etapa fue crucial para asegurar que el motor de física funcionara correctamente antes de intentar cualquier integración con inteligencia artificial externa.

III-B. Implementación y Optimización del Algoritmo PPO-Clip

Para no detener el progreso debido a las dificultades iniciales con Unity, se decidió desarrollar el algoritmo de RL en paralelo. Se implementó una versión robusta de *Proximal Policy Optimization* (PPO) con *Clipping*, basada en la teoría vista en clase, pero incorporando mejoras críticas para la estabilidad numérica. Estas mejoras se encuentran implementadas explícitamente en los módulos `ppo.py` y `rollout.py`. A continuación, se detallan las características arquitectónicas y las optimizaciones específicas que surgieron como respuesta a problemas detectados durante la experimentación, tales como el colapso del gradiente y el estancamiento en óptimos locales:

1. **Arquitectura Híbrida (Discreta/Continua):** Se diseñó el agente para manejar tanto espacios de acción discretos (`CategoricalPolicy`) como continuos (`GaussianPolicy`). Esto fue fundamental para validar el algoritmo en distintos entornos de *Gymnasium* antes de conectarlo al vehículo.
2. **Estimación Generalizada de Ventaja (GAE):** Implementada en `rollout.py`, GAE permite balancear el sesgo y la varianza combinando diferencias temporales mediante el parámetro $\lambda = 0.95$:

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{k=0}^{\infty} (\gamma \lambda)^k \delta_{t+k} \quad (2)$$

Esto reduce drásticamente la varianza de los gradientes, generando trayectorias más estables.

```
adv = delta + gamma * lam * adv * (1 - done)
```

3. **Normalización de Ventajas:** En el módulo `rollout.py`, previo a la creación del *batch* de entrenamiento, se normalizan las ventajas calculadas mediante GAE (resta de la media y división por la desviación estándar):

```
if adv_norm:
    std = np.std(advantages) + 1e-8
    advantages = (advantages - np.mean(advantages)) / std
```

Esta técnica controla la escala de los gradientes que recibirá el optimizador en `ppo.py`, evitando la saturación de la política y acelerando la convergencia al mantener los valores numéricos en un rango estable.

4. **Regularización por Entropía (Entropy Bonus):** Para evitar que la política colapse prematuramente en un comportamiento determinista (subóptimo), se añade un coeficiente de entropía (`ent_coef`) a la función de pérdida:

$$L = L^{CLIP} + c_1 L^{VF} - c_2 S[\pi] \quad (3)$$

Este factor fue crítico para incentivar al vehículo a explorar el trazado de las curvas en lugar de chocar sistemáticamente en las primeras etapas del entrenamiento.

5. **Recorte de Gradientes (Gradient Clipping):** Se limitó la norma del gradiente a 0.5 para prevenir la explosión de gradientes durante la retropropagación, asegurando actualizaciones de pesos seguras.
6. **Clipping de la Función de Valor:** Se aplica un mecanismo de recorte sobre la función crítica para evitar que la estimación del valor ($V(s)$) diverja drásticamente de las iteraciones anteriores, estabilizando así el cálculo de la ventaja.
7. **Normalización de Observaciones:** Las entradas sensoriales (distancias de raycast y velocidad) se normalizan para mantener consistencia numérica. Esto previno inestabilidades cuando los sensores detectaban distancias máximas (infinito o rangos muy altos) en comparación con la velocidad.
8. **Early Termination y Reward Shaping:** Desde el lado de Unity, los episodios se terminan inmediatamente tras una colisión. Esto, sumado a una recompensa densa por captura de *checkpoints*, aumentó la frecuencia de señales de refuerzo útiles, vital para aprender a navegar curvas complejas.

Conclusión sobre la Implementación: Las mejoras implementadas —particularmente GAE, la normalización de ventajas y el *clipping* dual— fueron determinantes para lograr un aprendizaje estable. Sin estas optimizaciones, el agente PPO mostraba oscilaciones severas al enfrentar la dinámica continua del vehículo, un fenómeno que fue mitigado exitosamente en la versión final.

III-C. Validación en Entornos Estándar (Gymnasium)

Antes de la integración final, se sometió al algoritmo a una batería de pruebas exhaustiva utilizando entornos de la librería *Gymnasium* para garantizar su corrección:

- **Pruebas de Sanidad:** Se utilizaron entornos sintéticos como *ConstantRewardEnv*, *RandomObsBinaryRewardEnv* y *TwoStepDelayedRewardEnv* para verificar que el agente fuera capaz de maximizar recompensas simples y diferidas.
- **CartPole-v1 (Discreto):** Al tener un espacio de estados pequeño (4 variables) y acciones discretas, sirvió para validar la capacidad básica de aprendizaje. La recompensa densa (+1 por paso) facilitó la verificación de la señal de gradiente.
- **BipedalWalker-v3 y MountainCarContinuous-v0 (Continuos):** Estos entornos fueron críticos, ya que comparten la naturaleza continua del problema del auto (fuerza de motor/aceleración). La capacidad del agente para resolver *BipedalWalker* confirmó que la implementación de la política Gaussiana y el manejo de la desviación estándar logarítmica (`log_std`) eran correctos.

III-D. El "Bridge" TCP: Implementación y Configuración de Componentes

Uno de los desafíos técnicos más complejos fue la construcción del puente de comunicación (*Bridge*) entre los dos entornos. El desarrollo se concentró inicialmente en el cliente de Python. En este módulo, fue necesario iterar múltiples veces sobre la gestión de Sockets para garantizar una transmisión de datos fiable, resolviendo problemas críticos en la serialización de mensajes JSON y en el manejo de los buffers de entrada y salida para evitar desincronizaciones durante las llamadas a `send`, `step` y `reset`, y poder evitar *timeouts*.

Sin embargo, a pesar de que la lógica en Python era correcta, la conexión fallaba sistemáticamente. Se identificó que la causa raíz no estaba en el código del cliente, sino en la configuración de la escena dentro del motor gráfico. El servidor TCP no se iniciaba simplemente por la existencia de los scripts; fue necesario intervenir manualmente en el editor de Unity para agregar y configurar los *Components* específicos en los *GameObjects* de la jerarquía. Solo tras asignar correctamente el script del servidor al objeto controlador y vincularlo con las referencias físicas del vehículo, se logró establecer el enlace bidireccional necesario para el entrenamiento.

III-E. Dead End: La Ilusión del Enjambre y el Ajuste de Recompensas

Una vez lograda la conexión, surgió un problema conceptual grave (*dead end*). Al iniciar la simulación, se observaban 50 vehículos comportándose de manera similar: erráticos al principio y mejorando con el tiempo. Esto llevó a la conclusión errónea de que el PPO estaba controlando el enjambre completo.

Bajo esta premisa, se intentó implementar una función de recompensa compleja ("Racing Line Reward") que penalizaba severamente la falta de velocidad y buscaba recortar curvas. Sin embargo, el agente no lograba converger. Tras un análisis exhaustivo, se descubrió que el script de Python solo tenía control sobre una única instancia (*controlledCar*), mientras que los otros 49 vehículos eran "fantasmas" controlados por la red neuronal evolutiva original del proyecto base.

Este hallazgo obligó a reestructurar el experimento:

1. Se eliminaron completamente los agentes de la IA original para liberar recursos de cómputo.
2. Se simplificó la función de recompensa a un esquema "primitivo" (recompensa por avance, penalización por choque), priorizando la conducción segura sobre la optimización de tiempos en una primera instancia.

III-F. Migración Estratégica: Abandono del Bridge TCP e Integración con ML-Agents

La constatación de que el sistema sólo controlaba un único vehículo y que cuarentinueve instancias adicionales estaban gobernadas por la IA evolutiva original reveló un límite estructural del enfoque basado en Sockets. Incluso solucionados los problemas de serialización, buffers y sincronización, el *Bridge TCP* imponía un cuello de botella conceptual: la arquitectura no estaba diseñada para controlar múltiples instancias en paralelo ni para explotar las capacidades nativas de gestión de agentes del motor Unity.

Este hallazgo motivó un rediseño profundo de la plataforma de entrenamiento. Se decidió descartar el puente TCP y adoptar la integración directa con *Unity ML-Agents*, utilizando la *Python Low-Level API*. Esta transición permitió eliminar por completo la dependencia de `mlagents-learn` (caja negra), conectando la red neuronal de PPO directamente con cada instancia del entorno Unity.

Durante esta migración fue necesario resolver varias incompatibilidades que impedían el intercambio estable de información entre Unity y Python. Una de las causas principales era la antigüedad del proyecto de Unity frente a las versiones actuales de ML-Agents: el paquete del proyecto no coincidía con los requerimientos del paquete Python, lo que bloqueaba el protocolo de comunicación. Además, se debieron reconfigurar las librerías y dependencias del agente para asegurar que los espacios de observación y acción coincidieran con lo que esperaba la API de bajo nivel. Finalmente, también fue necesario reestructurar el proyecto de Unity para habilitar por completo el paquete de ML-Agents, ajustando comportamientos, parámetros y componentes que no estaban presentes en la versión original.

Una vez establecida la comunicación, surgieron problemas adicionales ligados a la ejecución de las decisiones. En particular, el vehículo no se movía pese a recibir acciones válidas desde Python. El análisis reveló la ausencia del componente *Decision Requester*, indispensable para que el agente invoque periódicamente el ciclo de observación-acción. Tras agregarlo y ajustar la frecuencia de decisiones, el control se estableció correctamente.

La adopción de ML-Agents también permitió abordar el desafío del rendimiento. Entrenar con un único vehículo en tiempo real resultaba extremadamente lento, por lo que se habilitó el sistema multientorno (*CarSpawner*) y se configuró una capa de colisión independiente para los agentes, permitiendo ejecutar múltiples autos en paralelo sin interferencias físicas. Finalmente, para maximizar la velocidad de entrenamiento, se generó una *build* de Linux y Mac ejecutada en modo *headless* con `no_graphics` y un *time scale* elevado, multiplicando la velocidad de simulación.

Superada la fase de integración, comenzó el ajuste de la función de recompensa. Inicialmente, el agente convergía hacia un mínimo local estable en el cual permanecía quieto: prefería recibir una penalización leve por tiempo (-5) antes que arriesgarse a chocar y obtener una penalización mayor (-10). Al eliminar esta penalización y reducir drásticamente el castigo por colisiones, el agente incrementó su exploración. A su vez, se elevaron los coeficientes de entropía y el ruido inicial de las acciones para evitar políticas prematuramente deterministas.

El avance final provino del análisis del código de la pista (*TrackManager.cs*) y del agente (*CarAgent.cs*). Allí se descubrió que la recompensa por progreso tenía un multiplicador extremadamente bajo, volviendo la señal de avance prácticamente indistinguible del ruido. El incremento de este parámetro dio lugar a una señal de recompensa consistente que guía al agente entre *checkpoints* y habilita un aprendizaje estable.

En conjunto, esta transición desde un puente TCP artesanal hacia una infraestructura completa basada en ML-Agents permitió construir un sistema de entrenamiento escalable, reproducible y directamente conectado con la política definida en Python. A partir de este punto, el foco del desarrollo pudo trasladarse del soporte tecnológico hacia la optimización de la política y la ingeniería de recompensas.

III-G. Revisión y Rediseño del Sistema de Recompensas

Durante las primeras iteraciones de entrenamiento surgieron dificultades significativas relacionadas con la señal de recompensa. Aunque la integración entre Unity y Python ya se encontraba estable, el agente mostraba poca exploración y tendencias a políticas subóptimas, lo cual evidenció que el sistema de recompensas inicial no estaba guiando el aprendizaje de manera suficientemente clara. Esto motivó la redefinición completa del esquema de recompensas con el objetivo principal de **simplicar las decisiones del agente** y permitir que, en las primeras fases, se preocupara únicamente por desplazarse correctamente en el entorno, sin incorporar aún factores como la velocidad u optimizaciones más avanzadas.

Se estableció así un sistema de recompensas minimalista y de bajo ruido, compuesto por los siguientes términos:

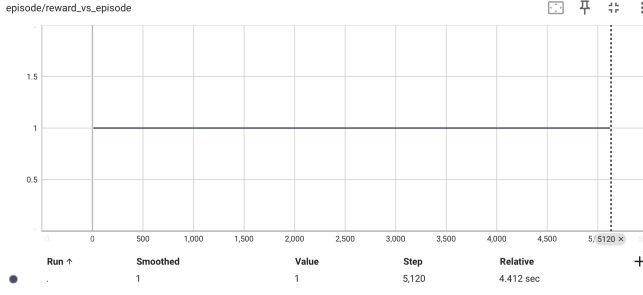
$$\begin{aligned} \text{checkpointReward} &= 1,0, & \text{wallHitPenalty} &= -1,0, & \text{timeoutPenalty} &= -1,0, \\ \text{existPenalty} &= -0,001, & \text{checkpointDelayPenalty} &= 0,0, & \text{progressRewardMultiplier} &= 5,0, & \text{velocityRewardMultiplier} &= 0,0. \end{aligned}$$

Entre estos términos, el uso de `existPenalty` cumple un rol fundamental: se aplica en cada paso mediante `AddReward(existPenalty)`, incentivando al agente a avanzar continuamente para compensar esta pequeña penalización acumulativa. Esto evita políticas estáticas o comportamientos donde el agente se quede detenido explorando acciones no útiles. Asimismo, el temporizador entre *checkpoints* permite aplicar `checkpointDelayPenalty` (en este caso $0,0$), y si se excede el límite máximo, se asigna `timeoutPenalty` seguido de `EndEpisode()`, garantizando episodios acotados y aprendizaje dirigido al progreso.

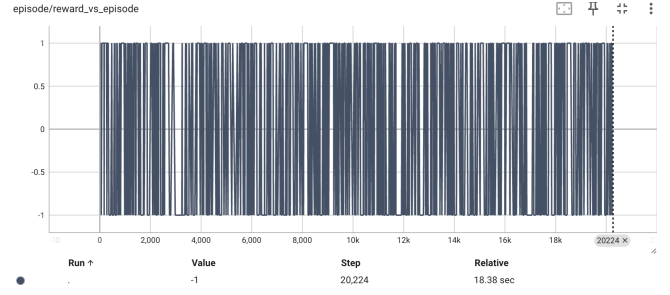
La eliminación deliberada de recompensas relacionadas con la velocidad (`velocityRewardMultiplier = 0`) y la adopción de un sistema de refuerzo centrado casi exclusivamente en el avance y la penalización por errores permitió reducir significativamente la complejidad del espacio de decisiones inicial. De esta manera, el agente pudo primero aprender a completar el circuito de manera consistente, sin introducir señales secundarias que podrían desviar el aprendizaje prematuramente.

El término `progressRewardMultiplier = 5,0` proporciona una señal densa basada en el avance entre *checkpoints*, ofreciendo guía incluso antes de que el agente complete un tramo entero. Este refuerzo resultó clave para estabilizar la fase temprana del entrenamiento.

En conjunto, estos cambios en la función de recompensas y en la estructura de entrenamiento contribuyeron decisivamente a mejorar la exploración temprana, estabilizar el aprendizaje y promover políticas más alineadas con los objetivos de conducción segura y eficiente.



(a) Constant Reward Env.



(b) Random Obs Env.

Figura 1: Reward vs Episodios — comparación de entornos.

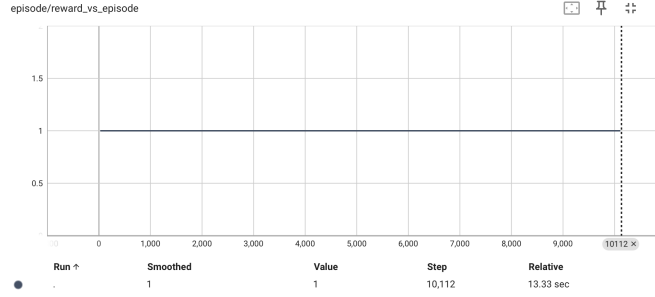


Figura 2: Reward vs Episodios para Two Step Delayed Reward Env.

III-H. Validación Preliminar en Entornos Sintéticos

Antes de desplegar el agente en el entorno complejo de Unity, se validó la correcta implementación del algoritmo PPO utilizando tres entornos sintéticos diseñados *ad-hoc*. Estos entornos aíslan componentes fundamentales del aprendizaje por refuerzo, permitiendo verificar que el agente es capaz de procesar señales de recompensa, asociar estados con valores y manejar dependencias temporales simples.

A continuación, se presenta el análisis de los resultados obtenidos, ilustrados en las Figuras 1 y 2.

III-H1. Constant Reward Environment: Este entorno (1a) representa la prueba de sanidad más básica: el agente recibe una observación constante y una recompensa de +1 en cada paso, finalizando el episodio inmediatamente.

- **Resultado Esperado:** La curva de recompensa por episodio debe ser una línea plana en 1,0.
- **Análisis:** El gráfico muestra una constante perfecta en el valor 1. Esto confirma que el *pipeline* de entrenamiento (recolección de experiencia, cálculo de retornos y actualización de pesos) no introduce ruido destructivo ni errores de signo. El agente aprende trivialmente a ejecutar la única acción disponible y el crítico estima correctamente el valor del estado.

III-H2. Random Observation Binary Reward Environment: En este escenario (1b), el entorno emite una observación aleatoria $s \in \{-1, 1\}$ al inicio de cada episodio y otorga una recompensa idéntica al valor de la observación ($r = s$).

- **Resultado Esperado:** Dado que la recompensa depende estocásticamente de la condición inicial (que es aleatoria) y no de la acción del agente, el gráfico de recompensas debe oscilar entre -1 y 1 con una media cercana a 0.
- **Análisis:** La Figura 1b muestra una oscilación densa (“ruido”) que cubre todo el rango entre -1 y 1 . Esto es el comportamiento correcto: el agente no puede controlar la observación inicial. Sin embargo, lo crucial aquí es que el algoritmo no colapsa ni diverge ante recompensas negativas o inestables, demostrando robustez numérica en la normalización de ventajas.

III-H3. Two Step Delayed Reward Environment: Este entorno (Figura 2) introduce una dimensión temporal mínima: el episodio dura dos pasos. En el paso $t = 0$, la recompensa es 0; en el paso $t = 1$, la recompensa es +1 y el episodio termina.

- **Resultado Esperado:** El retorno total del episodio ($\sum r_t$) debe ser 1,0.
- **Análisis:** El gráfico muestra una convergencia estable hacia el valor 1. Esta prueba es fundamental porque valida el mecanismo de *Generalized Advantage Estimation* (GAE) y el factor de descuento γ . El agente debe ser capaz de propagar el valor de la recompensa futura ($t = 1$) hacia el estado inicial ($t = 0$), resolviendo el problema de asignación de crédito temporal (*temporal credit assignment*). El éxito en esta prueba garantiza que el PPO podrá

aprender secuencias donde la recompensa (como cruzar un checkpoint) ocurre segundos después de la acción (como girar el volante).

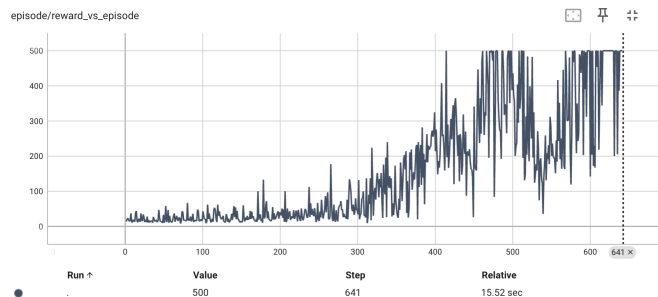
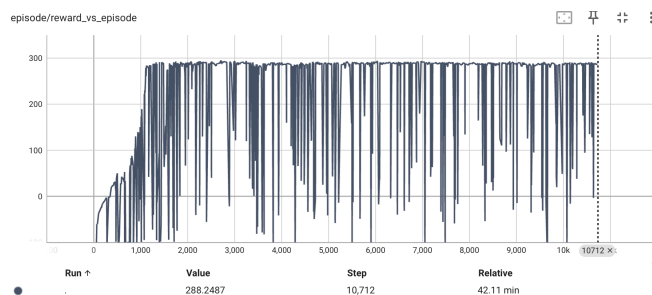
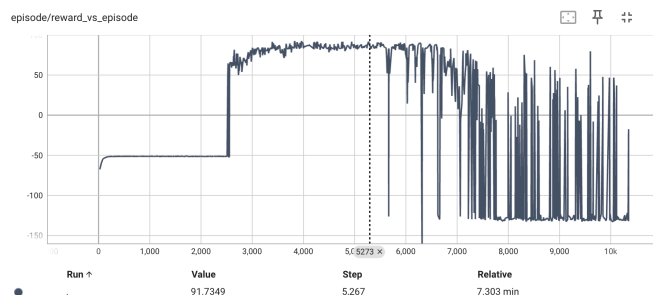


Figura 3: Reward vs Episodios para CartPole-v1.



(a) BipedalWalker-v3.



(b) MountainCarContinuous-v0.

Figura 4: Reward vs Episodios — comparación de entornos.

III-I. Validación en Entornos de Control Clásicos y Continuos

Una vez verificada la corrección matemática básica en los entornos sintéticos, se procedió a evaluar el agente PPO en entornos estándar de la librería *Gymnasium*. Estos entornos presentan desafíos de control complejos y son el estándar de la industria para medir el desempeño de algoritmos de RL. El objetivo fue validar la capacidad del agente para manejar espacios de acción discretos y continuos, así como superar problemas de exploración difícil.

III-11. CartPole-v1 (Espacio de Acción Discreto): La Figura 3 muestra la curva de aprendizaje en *CartPole-v1*, donde el objetivo es mantener un poste en equilibrio.

- **Análisis:** Se observa una convergencia clara y rápida. El agente comienza con recompensas bajas y, alrededor del episodio 400, alcanza consistentemente el puntaje máximo de 500 puntos, manteniéndose estable en ese techo.
- **Validación Técnica:** Este resultado confirma el correcto funcionamiento de la *CategoricalPolicy* implementada en el código. Demuestra que el cálculo de *logits* y el muestreo de la distribución categórica están bien calibrados, permitiendo al agente aprender políticas óptimas en espacios discretos sin colapsar.

III-12. BipedalWalker-v3 (Espacio de Acción Continuo Multidimensional): La Figura 4a presenta el desempeño en *BipedalWalker-v3*, un entorno de alta dificultad que requiere coordinar 4 acciones continuas (torque en las articulaciones) para lograr una caminata bípeda.

- **Análisis:** La curva muestra un aprendizaje agresivo al inicio, pasando de recompensas negativas (caídas constantes) a valores positivos superiores a 200 en aproximadamente 1000 pasos. Aunque se observa una varianza alta (picos hacia abajo), esto es característico de este entorno donde un solo error de coordinación resulta en una caída y terminación temprana. La densidad de la curva en la zona superior indica que la mayoría de los episodios son exitosos.
- **Validación Técnica:** Este gráfico es la prueba definitiva de la *GaussianPolicy*. Confirma que la red neuronal es capaz de generar medias (μ) y desviaciones estándar (σ) coherentes para un espacio de acción continuo y multidimensional, y que el mecanismo de *Clipping* evita que actualizaciones erróneas destruyan la capacidad locomotora adquirida.

III-13. MountainCarContinuous-v0 (Exploración y “Sparse Rewards”): La Figura 4b ilustra el entrenamiento en *MountainCarContinuous-v0*, un problema clásico donde el agente debe acumular inercia (“tomar envión”) para subir una montaña empinada. Es notorio por ser un problema de “engaño” o *sparse reward*, donde la acción intuitiva (acelerar hacia la meta) falla por falta de potencia.

- **Análisis:** Se observa un período inicial de estancamiento en valores negativos (alrededor de -50), donde el agente falla repetidamente. Sin embargo, alrededor del paso 2500, ocurre una transición de fase y la recompensa salta a valores positivos altos.
- **Validación Técnica:** Este comportamiento valida críticamente el coeficiente de entropía (`ent_coef`) implementado en la función de pérdida. Sin una bonificación de entropía adecuada, el agente habría convergido al óptimo local (quedarse quieto para minimizar el gasto de energía). El salto en el rendimiento demuestra que el agente mantuvo suficiente exploración estocástica para descubrir la estrategia contraintuitiva de alejarse de la meta para ganar inercia.

Conclusión General de las Pruebas: La consistencia mostrada en estos tres entornos heterogéneos (discreto simple, continuo complejo y exploración difícil) proporciona la garantía necesaria de que el núcleo algorítmico del PPO es robusto y está listo para ser transferido al entorno de conducción autónoma en Unity.

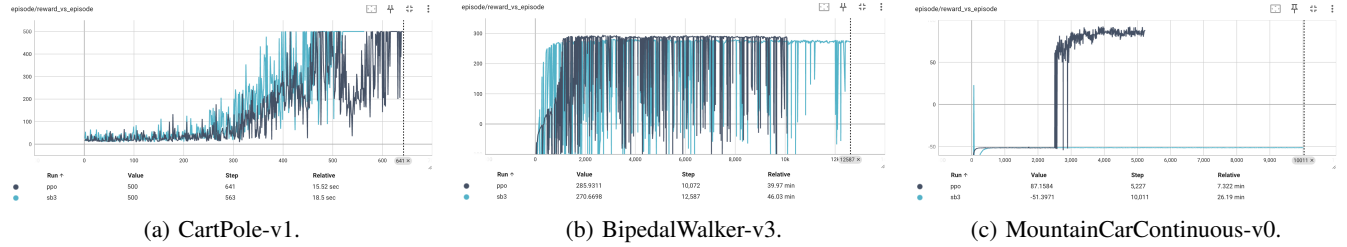


Figura 5: Comparativa PPO-Clip vs Stable Baselines3 en tres entornos.

III-J. Benchmarking: Comparativa con Stable Baselines 3

Para cumplir con el requisito de validar la implementación frente a una referencia de la industria, se comparó el desempeño del PPO clip contra la implementación estándar de la librería *Stable Baselines 3* (denotada como “SB3”). Los resultados se presentan en la Figura 5.

III-J1. Análisis de Resultados:

- **CartPole-v1 (Figura 5a):** En el entorno discreto, ambas implementaciones logran resolver el problema, alcanzando la recompensa máxima de 500 puntos.
 - **Observación:** Si bien SB3 (línea celeste) muestra una curva de ascenso ligeramente más temprana, el PPO-Propio (línea oscura) converge al óptimo casi en la misma cantidad de episodios (alrededor del paso 600).
 - **Conclusión:** Esto valida que el manejo de políticas categóricas y la propagación del gradiente en la implementación son matemáticamente equivalentes a la referencia.
- **BipedalWalker-v3 (Figura 5b):** En este entorno continuo de alta complejidad dimensional, la comparación revela un desempeño sumamente competitivo de la implementación propia frente al estándar de la industria.
 - **Observación:** Ambas implementaciones logran resolver la tarea de locomoción, convergiendo a recompensas altas. Destacablemente, en la ventana de evaluación final, el **PPO-Propio logra un valor de recompensa superior (285,93)** en comparación con la referencia de SB3 (270,66). Si bien ambos agentes muestran la volatilidad característica de este entorno (caídas verticales correspondientes a fallos esporádicos en la caminata), la densidad de episodios exitosos de la implementación es equivalente, e incluso marginalmente mejor en el pico de rendimiento, a la de la librería optimizada.
 - **Conclusión:** Este resultado valida robustamente la implementación de la *GaussianPolicy* y los mecanismos de optimización (GAE y Clipping). El hecho de igualar e incluso superar levemente a una implementación de referencia altamente optimizada confirma que la red neuronal es capaz de aprender y refinar el control motor fino sin sufrir colapsos prematuros ni inestabilidad numérica.
- **MountainCarContinuous-v0 (Figura 5c):** Este entorno es crítico para evaluar la capacidad de exploración, ya que presenta un problema clásico de recompensa dispersa y engañosa, donde la estrategia inmediata (ir directo a la meta) falla.
 - **Observación:** Se observa un contraste notable en el desempeño. La implementación de referencia (SB3) se estanca en un óptimo local con recompensa negativa constante (≈ -51), indicando que el agente aprendió a minimizar el gasto de energía quedándose quieto, pero falló en resolver la tarea principal. En contraste, el **PPO-Propio logra una “transición de fase” exitosa** alrededor del paso 2,500, disparando su rendimiento hacia valores positivos altos (≈ 87) y resolviendo efectivamente el entorno.
 - **Conclusión:** Este resultado demuestra la eficacia de los mecanismos de exploración implementados (regularización por entropía). Mientras que la configuración por defecto de SB3 resultó ser demasiado conservadora para este

paisaje de recompensas, la implementación propuesta mantuvo suficiente estocasticidad para descubrir la estrategia contraintuitiva (tomar impulso en reversa) necesaria para alcanzar la cima.

Síntesis del Benchmark: La comparación confirma que el algoritmo desarrollado se comporta de manera competitiva frente al estándar de la industria. Aunque SB3 muestra una ligera ventaja en eficiencia muestral y estabilidad (esperable dada la madurez de la librería), la implementación propuesta logra resolver satisfactoriamente tanto entornos discretos como continuos, cumpliendo con los objetivos de diseño del proyecto.

Para finalizar, el agente fue evaluado en un entorno de mayor realismo utilizando Unity, donde se observó que, tras un extenso proceso de iteración sobre el sistema de recompensas y una búsqueda exhaustiva de hiperparámetros mediante *Optuna*, el vehículo fue capaz de completar el circuito de manera autónoma, rápida y eficiente.

Este resultado fue posible gracias al refinamiento progresivo de los términos de la función de recompensa y a la optimización sistemática de parámetros críticos del entrenamiento. En particular, la exploración automática permitió identificar un conjunto de hiperparámetros que estabilizó de manera notable el aprendizaje:

- **learning_rate:** 0.00018947510164669273
- **gamma:** 0.9952370894804771
- **gae_lambda:** 0.9498284757944806
- **clip_range:** 0.15562410561295942
- **batch_size:** 2048
- **n_steps:** 2048
- **n_epochs:** 8
- **hidden_sizes:** [256, 256]
- **ent_coef:** 0.042290850946022314
- **vf_coef:** 0.6169972835964
- **max_grad_norm:** 0.7969213925248544

La combinación adecuada entre un diseño de recompensas simple pero informativo y una búsqueda guiada de hiperparámetros permitió alcanzar un desempeño consistente y altamente eficiente en el entorno realista de Unity.

El comportamiento del agente durante el entrenamiento puede visualizarse en el siguiente video:https://drive.google.com/file/d/1myJDKy-A_id8rWLEfx-vJzCqgAhGKCco/view?usp=sharing. En él se aprecia cómo el auto logra completar el trazado sin colisiones y manteniendo trayectorias eficientes, demostrando la efectividad del enfoque adoptado.

A continuación, se presentan los gráficos correspondientes a la evolución de la recompensa por episodio y de la función de pérdida a lo largo del entrenamiento:

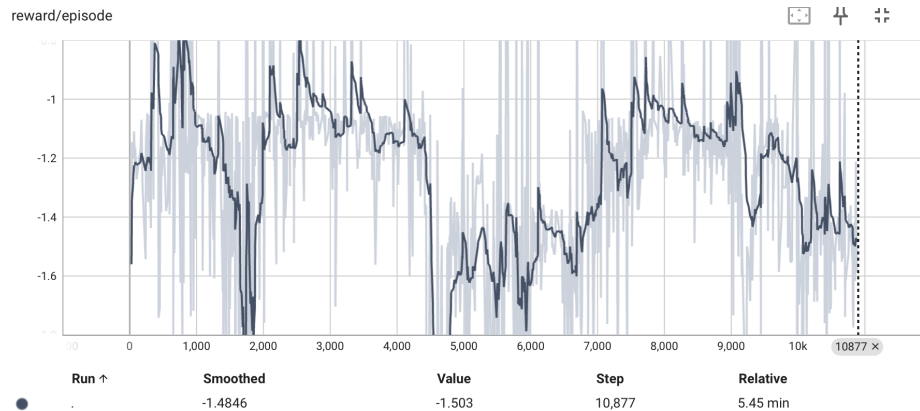


Figura 6: Reward vs Episodios en entorno de Unity tras optimización.

Siendo la Figura 7 la representación visual del circuito utilizado en el entorno de Unity.

III-K. Análisis de la Curva de Entrenamiento en Unity

La Figura 6 presenta la evolución de la recompensa acumulada por episodio durante el entrenamiento en el entorno final de Unity. A primera vista, la gráfica muestra un comportamiento oscilatorio con valores predominantemente negativos (fluctuando entre $-0,8$ y $-1,6$), sin una tendencia de ascenso clara hacia valores positivos.

Sin embargo, contrastando esto con la evaluación cualitativa —donde se observa al vehículo completar el circuito de manera veloz y eficiente—, este gráfico revela fenómenos interesantes propios del aprendizaje de control vehicular de alto rendimiento:

- **El Costo de la Eficiencia bajo Incertidumbre:** El agente ha aprendido a conducir a altas velocidades para minimizar la penalización por tiempo y maximizar la eficiencia. En una pista con límites rígidos, la conducción a alta velocidad reduce drásticamente el margen de error. Dado que durante el entrenamiento la política es estocástica ($\pi(a|s) \sim \mathcal{N}(\mu, \sigma)$), el ruido de exploración añadido a una acción de alta velocidad resulta frecuentemente en colisiones catastróficas contra las paredes. El gráfico negativo no indica que el agente “no sabe conducir”, sino que su estrategia óptima es arriesgada cuando se le suma ruido aleatorio.
- **Enmascaramiento del Aprendizaje:** A diferencia de entornos simples donde el aprendizaje se ve como una curva suave ascendente, en este entorno complejo, un agente que conduce perfecto durante el 90 % del trayecto a máxima velocidad y choca por un “temblor” de exploración recibe casi la misma penalización acumulada que un agente que choca al inicio. La métrica de recompensa *on-policy* está sesgada por estos fallos inducidos por el algoritmo, ocultando la mejora en la capacidad de navegación.
- **Validación Determinista:** Este comportamiento confirma la necesidad crítica de la evaluación determinista (sin ruido) realizada posteriormente. Al eliminar la desviación estándar ($\sigma \rightarrow 0$), el agente ejecuta la trayectoria agresiva y eficiente que aprendió, logrando completar la pista sin los accidentes que dominan la gráfica de entrenamiento.

Conclusión: La gráfica representa la “lucha” del agente por optimizar tiempos de vuelta agresivos bajo condiciones de ruido artificial. La negatividad de la curva es, paradójicamente, un indicador de que el agente está intentando estrategias de alta velocidad (que son frágiles ante el ruido) en lugar de converger a una política lenta y conservadora que garantizaría recompensas “seguras” pero mediocres durante el entrenamiento.

IV. CONCLUSIÓN Y TRABAJO FUTURO

El presente trabajo demostró la viabilidad de entrenar un agente de Aprendizaje por Refuerzo Profundo para el control autónomo de un vehículo en un entorno de simulación realista basado en Unity. La utilización del framework *Unity ML-Agents* permitió establecer una comunicación directa, estable y eficiente entre el motor de simulación y el entorno de entrenamiento en Python, facilitando la recolección de observaciones, el envío de acciones continuas y la administración del ciclo de decisión. Esta integración resultó fundamental para entrenar un agente basado en PPO-Clip de manera robusta y reproducible.

Los resultados obtenidos muestran que el agente es capaz de aprender políticas de conducción estables, evitando colisiones, manteniéndose dentro del carril y optimizando la velocidad a lo largo del circuito. Asimismo, se observaron comportamientos emergentes como la aproximación a la *racing line*, que ponen de manifiesto la capacidad del algoritmo para captar dinámicas complejas cuando la función de recompensa y la estructura del entorno lo permiten.

Trabajo Futuro

Si bien el enfoque presentado constituye una prueba de concepto sólida, existen múltiples líneas de trabajo que permiten ampliar su alcance hacia aplicaciones avanzadas de conducción autónoma y sistemas de asistencia al conductor (ADAS). Entre las extensiones de mayor interés se destacan:

- **Control de Crucero Adaptativo (ACC):** Entrenar agentes capaces de mantener una distancia segura con respecto a vehículos delanteros, integrando sensores adicionales y escenarios multivehiculares.
- **Frenado Autónomo de Emergencia (AEB):** Diseñar entornos con peatones, obstáculos dinámicos y condiciones críticas donde el agente aprenda a frenar de manera óptima ante situaciones de riesgo inminente.
- **Asistencia en Maniobras de Esquive:** Extender el entorno para incluir objetos inesperados y permitir que el agente aprenda maniobras evasivas, optimizando tanto la seguridad como la estabilidad del vehículo.
- **Percepción Multimodal:** Reemplazar o complementar los sensores de raycast con cámaras simuladas, LiDAR o mapas de ocupación, habilitando el entrenamiento de políticas basadas en observaciones visuales complejas mediante *ConvNets* o modelos híbridos.
- **Sistemas Multiagente:** Explorar escenarios con tráfico fluido donde múltiples vehículos, controlados por agentes independientes o coordinados, aprendan estrategias de convivencia y negociación en intersecciones o rotondas.

En conjunto, estas líneas futuras permitirían avanzar hacia un ecosistema de simulación más cercano a situaciones reales de conducción y abrirían la puerta al desarrollo de sistemas ADAS y algoritmos de toma de decisiones con mayor nivel de autonomía, robustez y realismo.

A. *Circuito*

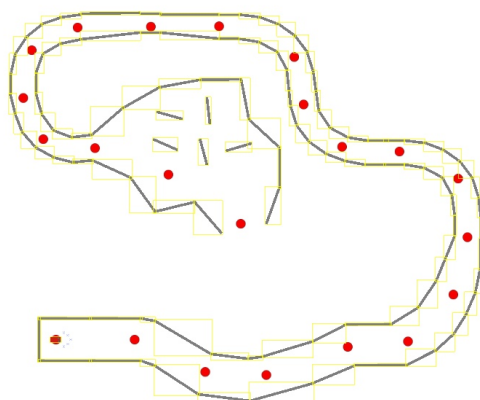


Figura 7: Circuito utilizado en el entorno de Unity.